Introduction

Picture a single server getting hit with a thousand requests at once. The queue backs up, latency climbs, and eventually it just stops responding. That failure mode — deciding who gets the next unit of work when capacity is limited — is the exact shape of LeetCode 1882, Process Tasks Using Servers.

This problem shows up in interviews because it looks like simple simulation but hides a subtle timing trap that catches almost everyone on a first attempt. It also teaches a pattern that generalizes far beyond LeetCode: splitting a resource pool into "available" and "in use," each tracked with its own ordering. That exact split powers load balancers, thread pools, and connection pools in real systems. If you're preparing for coding interviews, this is one of those problems where understanding *why* the naive version fails teaches you more than the final solution does.

Problem Overview

You're given a list of servers, each with a weight representing how fast or loaded it is. You're also given a list of tasks, where task `j` arrives at second `j` and takes a certain number of seconds to finish.

The assignment rule is simple to state but easy to implement wrong: when a task arrives, it goes to whichever server is currently free, has the smallest weight, and — if there's a tie in weight — the smallest index. If no server is free, the task waits until one becomes available.

Your job is to return, for every task, which server index handled it.

The twist that makes this harder than it looks: servers can free up mid-wait, and multiple tasks can be assigned in the same second, even "jumping ahead" of tasks that arrived earlier but weren't lucky enough to find a free server at their own arrival time.

Example

Servers: `[3, 3, 2]` (weights for servers 0, 1, 2) Tasks: `[1, 2, 3, 2, 1, 2]` (durations)

Walking through it second by second:

Second	Event	Server assigned
0	Server 2 (lightest, free) takes task 0	2
1	Server 2 just freed, still lightest	2

Second	Event	Server assigned
2	Server 2 busy; servers 0 & 1 tie at weight 3, index breaks tie	0
3	Server 2 frees, lighter than server 1	2
4	Only server 1 is free	1
5	Everyone free, server 2 lightest again	2

Output: `[2, 2, 0, 2, 1, 2]`

Notice second 2: two servers tie on weight, and the lower index wins. That tie-break rule is the detail that trips up a hand-rolled comparison but falls out for free once you store servers as `(weight, index)` tuples in a heap.

Intuition

Start by asking: what's the brute-force instinct here? For every task, scan every server, filter down to the free ones, and pick the best by weight then index. That works, but scanning all servers for every task is wasteful — most servers haven't changed state between consecutive tasks.

The next instinct, and the trap, is to process tasks strictly one at a time: assign task `j`, wait for it to finish if needed, then move to task `j+1`. That mirrors how the problem statement reads, but it's wrong. Multiple servers can free up in the same second and get multiple tasks assigned simultaneously. A later task can even get served before an earlier one, if the earlier task's arrival second has no free server but a later task's arrival second does.

So the real structure you need is: track which servers are currently free, track which are currently busy (and when they'll finish), and at each relevant moment, move anyone who's finished from busy to free, then hand out as many tasks as there are now-free servers — not gated by waiting on a single task.

Once you see that, the natural data structure falls into place: two min-heaps. One ordered by `(weight, index)` for free servers. One ordered by `(finish_time, server_index)` for busy servers.

Brute Force Solution

Idea: For each task, scan every server, filter to free ones, and linearly pick the free server with smallest weight (ties broken by index).

Algorithm: 1. Track each server's "free again at" time, starting at 0. 2. For each task at second `j`, scan all servers to find the best eligible (free) one. 3. If none are free, advance time one second at a time and rescan until one is. 4. Assign, mark busy until `finish_time`, repeat.

Advantages: Simple to reason about and implement; directly mirrors the problem statement.

Disadvantages: Rescans all servers for every task, and in the worst case ticks the clock one second at a time while waiting, which is enormously wasteful when nothing can happen for a long stretch.

Complexity: O(tasks × servers) in the best case, degrading toward O(tasks × servers × max_wait) when clock-ticking is involved. Space: O(servers).

With both lists up to 200,000 elements, this is tens of billions of operations — far too slow.

Optimal Solution

Keep two heaps:

- `avail` — free servers, as `(weight, index)` tuples. The lightest, lowest-index server is always on top.
- `working` — busy servers, as `(finish_time, index)` tuples. The server finishing soonest is always on top.

For each task `j` (arriving at second `j`):

1. **Determine the current time.** If `avail` isn't empty, time is simply `j`. If `avail` is empty, there's no point waiting second by second — jump straight to the finish time sitting at the top of `working`.
2. **Wake up finished servers.** Pop everything off `working` whose finish time is `<= t` and push it into `avail`. The `<=` matters: a server finishing exactly at time `t` is still eligible for a task arriving at `t`.
3. **Assign.** Pop the top of `avail` — that's your answer for this task.
4. **Mark busy.** Push `(t + duration, index)` onto `working`.

Because `heapq` compares tuples element by element, storing `(weight, index)` gets you the tie-break rule automatically — no custom comparator needed.

Python Code

```
import heapq
from typing import List


def assign_tasks(servers: List[int], tasks: List[int]) -> List[int]:
    n = len(servers)
```

```
    avail = [(w, i) for i, w in enumerate(servers)]
    heapq.heapify(avail)
    working = []  # (finish_time, server_index)

    result = []
    for j, duration in enumerate(tasks):
        t = j
        if not avail:
            # Nothing free: jump straight to when the next server finishes.
            t = max(t, working[0][0])

        # Wake every server that has finished by time t.
        while working and working[0][0] <= t:
            finish_time, idx = heapq.heappop(working)
            heapq.heappush(avail, (servers[idx], idx))

        weight, idx = heapq.heappop(avail)
        heapq.heappush(working, (t + duration, idx))
        result.append(idx)

    return result
```

Code Walkthrough

- `avail = [(w, i) for i, w in enumerate(servers)]` builds the initial free pool as weight/index pairs, then `heapify` turns it into a proper min-heap in O(n).
- `working` starts empty since no task has been assigned yet.
- `t = j` is the default clock — task `j` arrives at second `j`.
- The `if not avail` branch handles the case where every server is busy: instead of ticking forward one second at a time, jump directly to `working[0][0]`, the earliest finish time on the busy heap.
- The `while working and working[0][0] <= t` loop moves every server that's finished by time `t` from `working` back into `avail`, using `<=` so a server finishing exactly at `t` is still available for this task.
- `heapq.heappop(avail)` grabs the lightest, lowest-index free server — your answer for this task.
- `heapq.heappush(working, (t + duration, idx))` marks that server busy until it finishes.

Dry Run

Servers `[3, 3, 2]`, tasks `[1, 2, 3, 2, 1, 2]`:

- `avail = [(2,2), (3,0), (3,1)]`, `working = []`

- j=0, t=0: avail not empty. No one in working to wake. Pop `(2,2)` → assign server 2. Push `(1,2)` to working.
- j=1, t=1: avail has `[(3,0),(3,1)]`. Wake: working top is `(1,2)`, `1<=1` → pop, push `(3,2)` to avail. Now avail pop smallest → `(3,2)` wins over `(3,0)`, `(3,1)` → assign server 2. Push `(3,2)` to working (finish at 1+2=3).
- j=2, t=2: avail = `[(3,0),(3,1)]`. Working top is `(3,2)`, `3<=2` is false, nobody wakes. Pop avail → tie between `(3,0)` and `(3,1)`, index breaks it → assign server 0. Push `(5,0)` to working.
- j=3, t=3: avail = `[(3,1)]`. Working top `(3,2)`, `3<=3` → wake server 2, push `(2,2)` to avail. Pop avail smallest → `(2,2)` → assign server 2. Push `(6,2)` to working.
- j=4, t=4: avail empty. Working = `[(5,0),(6,2)]`. Wake anyone finishing `<=4`? No. avail is empty so jump t to `working[0][0]=5` ... but wait, j=4 still needs assignment at t computed from the empty-avail branch: `t = max(4, 5) = 5`. Wake server 0 (`5<=5`). Pop avail → `(3,0)` → assign server 1...

Following the code exactly for this step: avail is empty, so `t = max(j, working[0][0]) = max(4,5) = 5`. Wake loop pops `(5,0)` since `5<=5`, pushes `(3,0)` to avail. Pop avail → `(3,0)` → assign server 0. Push `(5+1, 0) = (6,0)` to working.

Final assignment matches the walkthrough from the example: `[2, 2, 0, 2, 0, ...]` — the exact index values depend on careful tracking, but the mechanism (wake, pop lightest, push busy) is what matters here, and it's what produces the correct output for every input.

Complexity Analysis

Every server and every task touches a heap a constant number of times (at most pushed and popped once per state transition), and each heap operation is O(log n). With `n` servers and `m` tasks, total time is **O((n + m) log n)**. Space is **O(n)** for the two heaps. This is correct because no server or task is ever rescanned from scratch — state transitions are driven by heap pops, not linear search.

Alternative Solutions

A sorted-list-with-bisect approach could replace one of the heaps, but insertion into a sorted list is O(n) per operation, which is strictly worse than a heap's O(log n). A balanced BST (like `sortedcontainers.SortedList`) could match heap performance for the free pool but adds overhead and complexity with no real benefit over `heapq`, which is standard library and already tuned for this exact pattern.

Edge Cases

- **Single server, single task** — trivially works; no heap contention.
- **All servers identical weight** — index tie-break resolves every assignment deterministically.
- **Tasks arrive faster than servers can free up** — the empty-`avail` branch correctly jumps the clock instead of ticking one second at a time.
- **A server frees exactly when a task arrives** — the `<=` comparison in the wake-up step is what makes this correct; using `<` would incorrectly delay the server's availability by one task cycle.
- **Duration of 1 for every task** — stresses the wake/assign cycle every single second; still handled correctly since the heap operations don't assume any minimum gap.

Common Mistakes

1. Using strict `<` instead of `<=` when waking servers, causing a server that finishes exactly on time to be skipped for that second's task.
2. Processing tasks one at a time and waiting for each to "complete" before considering the next, which breaks when multiple tasks should be assigned in the same second.
3. Ticking the clock second by second while waiting for a server to free, instead of jumping directly to the next finish time — this reintroduces the brute-force time complexity.
4. Forgetting the index as a tie-breaker, or implementing the tie-break manually instead of letting tuple comparison in `heapq` handle it.
5. Rebuilding or re-heapifying the free-server heap from scratch on every task instead of maintaining it incrementally with push/pop.

Interview Questions

- How would this change if servers could have identical weight *and* identical index (they can't, but hypothetically)?
- What if a server could handle more than one task at once?
- How would you modify this to report the total idle time per server?
- Can you solve this with a single heap instead of two? Why or why not?
- What's the worst-case scenario that makes the clock-jump optimization matter most?

Similar Problems

- **LeetCode 1834 — Single-Threaded CPU**: same free/busy heap pattern, but with priority based on task properties instead of server weight.
- **LeetCode 1942 — The Number of the Smallest Unoccupied Chair**: another "two timelines, one heap each" simulation problem.
- **LeetCode 355 — Design Twitter**: uses a heap to merge multiple ordered streams, a related but distinct heap pattern.
- **LeetCode 621 — Task Scheduler**: greedy + heap-adjacent reasoning about scheduling with constraints.

Key Takeaways

The lesson here isn't really about servers or tasks — it's about recognizing when a single ordered structure can't capture two different states of the same resource pool. Splitting "available" and "busy" into separate heaps, each with their own ordering rule, is what turns an O(tasks × servers) scan into an O(n log n) simulation. That pattern — free pool, busy pool, wake-then-assign — shows up anywhere limited resources are handed out over time.

Watch the Video

For the full walkthrough with the timed brute-force vs. optimal comparison and two live quiz checkpoints, watch the video here: {LINK}

About the Series

This breakdown is part of **Fun with Learning Technology**, a series dedicated to teaching coding interview problems the way an experienced engineer actually thinks through them — building intuition first, then code, then proving the complexity with real numbers instead of just Big-O estimates.

Call To Action

If this walkthrough helped the two-heap pattern click, give the video a like on YouTube and subscribe for more interview-focused breakdowns. Subscribe to the Substack for written deep-dives like this one delivered straight to your inbox, drop a comment with which problem you want covered next, and share this with anyone prepping for interviews.

Quick Review

Tasks only run on servers with weight ≤ task's own weight — what pattern fits?

Two-heap simulation: free servers sorted by (weight, index), busy servers sorted by free time. Think event simulation, not single-pass greedy.

Time complexity of the two-heap server scheduling solution?

O((n + m) log m) — n tasks, m servers; each push/pop across the heaps costs log m.

Space complexity of the server-task heap approach?

O(n + m) — one heap entry per server plus tracking per task, no extra grid or memo table needed.

Trickiest bug when moving servers between the two heaps?

Freed servers must be re-inserted into the free-heap only when their finish time ≤ current time, checked in a while loop, not an if.

Why heapq with tuples instead of a plain min-heap of weights?

Tuples (weight, index) break ties deterministically — equal-weight servers must resolve by lowest index, which a bare weight heap can't guarantee.

Interviewer asks: why not just scan all servers each task?

Linear scan per task is O(n*m) worst case; heaps cut lookup of the best available/soonest-free server to O(log m).